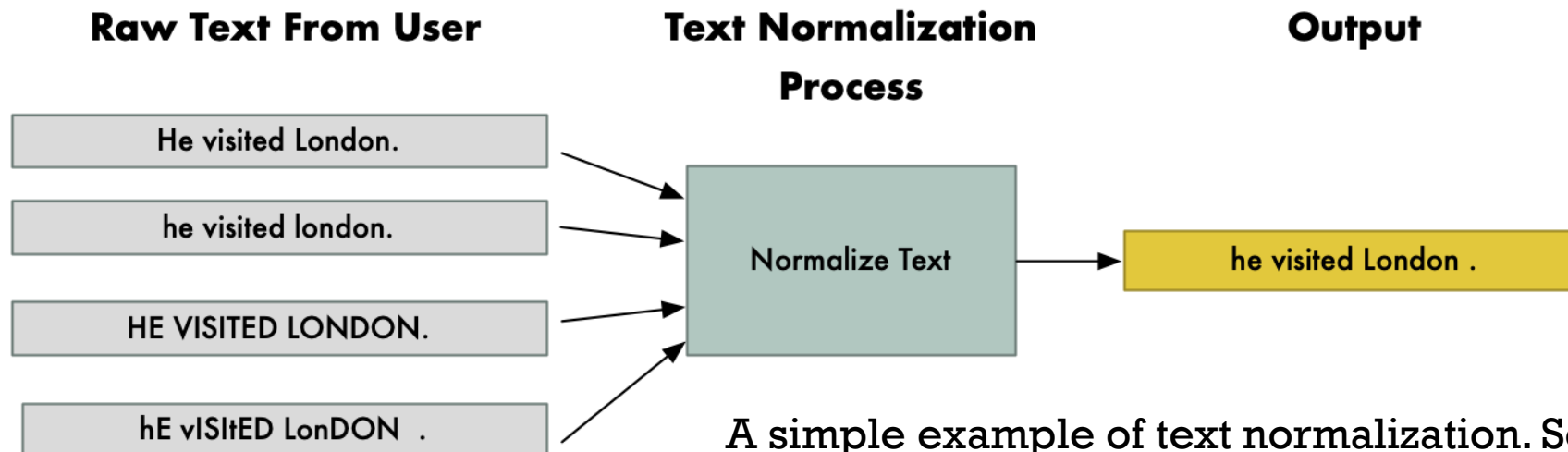


TEXT NORMALIZATION 1: TOKENIZATION

Natural Language Processing

Text Normalization

- Every NLP task requires text normalization:
 - Tokenizing (segmenting) words
 - Normalizing word formats
 - Segmenting sentences



A simple example of text normalization. Source: Geitgey 2020.
<https://devopedia.org/text-normalization>

Tokenization

- Tokenization is the first step in any NLP pipeline.
- **Tokenization** converts a sequence of text into smaller (meaningful) units, called **tokens**, for easier machine analysis, helping machines understand human language.
- Tokens are the building blocks of Natural Language.
- Common types of tokens are sentences, words, characters, and subwords.
- Tokenization is important for:
 - Feature extraction
 - Effective Text Processing and Analysis
 - Language Modelling
 - Vocabulary Management

Types of Tokenization

- **Sentence Tokenization:** The text is segmented into sentences during sentence tokenization.

Input: "Tokenization is an important NLP task. It helps break down text into smaller units."

Output: ["Tokenization is an important NLP task.", "It helps break down text into smaller units."]

Types of Tokenization

- **Word Tokenization:** This method breaks text down into individual words. It's the most common approach and is particularly effective for languages with clear word boundaries like English.

Input: "Tokenization is an important NLP task."

Output: ["Tokenization", "is", "an", "important", "NLP", "task", "."]

Types of Tokenization

- **Character Tokenization:** The text is segmented into individual characters.

Input: "Tokenization"

Output: ["T", "o", "k", "e", "n", "i", "z", "a", "t", "i", "o", "n"]

Types of Tokenization

- **Subword Tokenization:** This method breaks text into units that might be larger than a single character but smaller than a full word. This approach is especially useful for languages that form meaning by combining smaller units or when dealing with out-of-vocabulary words in NLP tasks.

Input: "unusual"

Output: ["un", "usual"]

Input: "chatbots"

Output: ["chat", "bots"]

Types of Tokenization

- **Phrase Tokenization:** We can also tokenize text into phrases or chunks that convey a specific meaning. For instance, the phrase “Los Angeles” might be a single token instead of two separate words.
- **Number Tokenization:** Number tokenization uses digits as the primary component of the token and segments numbers from the rest of the body of text.

Input: "Los Angeles is the second-most populous city in the US"

Output: ["Los Angeles", "is", "the", "second-most", "populous", "city", "in", "the", "US"]

Input: "She had 15 cats"

Output: ["15"]

Issues in Tokenization

- Can't just blindly remove punctuation:
 - [m.p.h.](#), [Ph.D.](#), [AT&T](#), [cap'n](#)
 - prices ([\\$45.55](#))
 - dates ([01/02/06](#))
 - URLs (<http://www.stanford.edu>)
 - hashtags ([#nlproc](#))
 - email addresses (someone@cs.colorado.edu)
- Clitic: a word that doesn't stand on its own
 - ["are"](#) in [we're](#), French ["je"](#) in [j'ai](#), ["le"](#) in [l'honneur](#)
- When should multiword expressions (MWE) be words?
 - [New York](#), [rock 'n' roll](#)

Subword Tokenization

- We can use our data to automatically tell us what the tokens should be.
- This is especially useful in dealing with unknown words, an important problem in language processing.
- To deal with this unknown word problem, modern tokenizers automatically induce sets of tokens that include tokens smaller than words, called subwords.
- Subwords can be arbitrary substrings, or they can be meaning-bearing units like morphemes.
- A morpheme is the smallest meaning-bearing unit of a language; for example, the word *unwashable* has the morphemes un-, wash, and -able.

Subword Tokenization

- Three common algorithms:
 - **Byte-Pair Encoding (BPE)** (Sennrich et al., 2016)
 - **Unigram language modeling tokenization** (Kudo, 2018)
 - **WordPiece** (Schuster and Nakajima, 2012)
- Most tokenization schemes have 2 parts:
 - A **token learner** that takes a raw training corpus and induces a vocabulary (a set of tokens).
 - A **token segmenter** that takes a raw test sentence and tokenizes it according to that vocabulary

Byte Pair Encoding (BPE) token learner

- Let vocabulary be the set of all individual characters
= {A, B, C, D,..., a, b, c, d....}

Repeat:

Choose the two symbols that are most frequently adjacent in the training corpus (say 'A', 'B')

Add a new merged symbol 'AB' to the vocabulary

Replace every adjacent 'A' 'B' in the corpus with 'AB'.

Until k merges have been done.

BPE token learner algorithm

function BYTE-PAIR ENCODING(strings C , number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters

for $i = 1$ **to** k **do** # merge tokens til k times

$t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C

$t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating

$V \leftarrow V + t_{NEW}$ # update the vocabulary

 Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus

return V

Byte Pair Encoding (BPE) Addendum

Most subword algorithms are run inside space-separated tokens.

So we commonly first add a special end-of-word symbol '___' before space in training corpus

Next, separate into letters.

BPE token learner

Original (very fascinating 🤖) corpus:

low low low low low lowest lowest newer newer newer
newer newer newer wider wider wider new new

Add end-of-word tokens, resulting in this vocabulary:

vocabulary

—, d, e, i, l, n, o, r, s, t, w

BPE token learner

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w

Merge **e r** to **er**

corpus

5 l o w _
2 l o w e s t _
6 n e w er _
3 w i d er _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er

BPE token learner

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er

Merge **er _** to **er_**

corpus

5 l o w _
2 l o w e s t _
6 n e w e r_
3 w i d e r_
2 n e w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er

BPE token learner

corpus

5 l o w _
2 l o w e s t _
6 n e w er_
3 w i d er_
2 n e w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er

Merge **n** **e** to **ne**

corpus

5 l o w _
2 l o w e s t _
6 ne w er_
3 w i d er_
2 ne w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er, ne

BPE token learner

The next merges are:

Merge	Current Vocabulary
(ne, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new
(l, o)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo
(lo, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low
(new, er_)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_
(low, _)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_

BPE token segmenter algorithm

On the test data, run each merge learned from the training data:

- Greedily
- In the order we learned them
- (test frequencies don't play a role)

So: merge every **e r** to **er**, then merge **er _** to **er_**, etc.

- Result:
 - Test set "n e w e r _" would be tokenized as a full word
 - Test set "l o w e r _" would be two tokens: "low er_"

Properties of BPE tokens

Usually include frequent words

And frequent subwords

- Which are often morphemes like *-est* or *-er*

References

- Daniel Jurafsky and James H. Martin. 2024. Speech and Language Processing, 3rd edition.
- <https://www.coursera.org/articles/tokenization-nlp>
- <https://www.datacamp.com/blog/what-is-tokenization>
- <https://www.geeksforgeeks.org/nlp-how-tokenizing-text-sentence-words-works/>
- <https://www.analyticsvidhya.com/blog/2020/05/what-is-tokenization-nlp/>